

Problem Set 03 — The Long Line

Corresponds to: 03-phylogeny — Geographies, characters, and the history of the compiler idea.

This problem set asks you to go outside the code and into history. Most of the materials you need are freely available. Some require library access. All are worth finding.

Allow yourself four to five hours, spread across a week.

Problem 1 — Thompson's question [reading + writing, 45 min]

Find "Reflections on Trusting Trust" by Ken Thompson (1984). It is eight pages, published in *Communications of the ACM* vol. 27, no. 8. It is widely available online — search the exact title.

Read the whole thing. Then answer:

Part A: What is the "trusting trust" attack? Explain it step by step, in your own words, as if explaining to someone who hasn't read the paper. The explanation should cover: (1) the Trojan horse in the compiler, (2) why fixing the source code doesn't fix the binary, (3) what it would take to actually fix it.

Part B: You have TCC's source in `practice/tcc/`. You can read every line. Can you verify that TCC does not contain a trusting trust attack? Explain your reasoning carefully. This is not a trick question with a clean answer.

Write at least 400 words total across both parts.

Problem 2 — Before C [research, 25 min]

Ritchie's 1993 paper describes B, the language that preceded C. Find it (from Problem 6 of PS-01) and read the section on B. Then find the "Users' Reference to B" by Ken Thompson (1972), available through the Bell Labs archives or tuhs.org.

Write down:

What were the three most significant things C added that B lacked?

For each, explain why the feature matters specifically for writing an operating system kernel. (Think: what does a kernel do that needs this?)

Ritchie says C's type system was partly motivated by PDP-11 hardware. What does he mean?

Problem 3 — Hopper's claim [research, 30 min]

Grace Hopper said: *"Nobody believed that I had a running compiler and nobody would touch it. They told me computers could only do arithmetic."*

Find a primary source — a paper, oral history, or interview by or about Hopper. The Computer History Museum (computerhistory.org) has oral history interviews. The ACM Digital Library has her 1952 paper "The Education of a Computer."

Write down:

What was the specific technical or conceptual objection to her claim? Why did her colleagues believe compilers were impossible?

Why were they wrong? What assumption did they make that turned out not to hold?

Why was it hard to see they were wrong at the time?

This is a question about how new ideas fail to be believed. Think carefully.

Problem 4 — Geography exercise [analysis, 25 min]

For each of the five geographies in the phylogeny document, trace a specific connection to something in `practice/tcc/` or `02/gcc/`.

- **Bletchley Park / Alan Turing:** The concept that makes any of this possible. What specific property of C programs or TCC's compilation process depends on Turing's 1936 result?
- **Bell Labs:** The most direct connection. Where in `tcc.c` or `tccgen.c` does Ritchie's original design still show up?
- **Berkeley / BSD:** Find a BSD-licensed file or a file with a BSD-style copyright notice in `02/tcc/` or `02/gcc/`. What is it?
- **MIT / Stallman:** Read the license header at the top of `practice/tcc/tcc.c`. What license is it? What does that license require of you if you modify and distribute TCC?
- **Helsinki / Torvalds:** TCC is commonly used on Linux. What assumption does TCC make in its Linux-specific code about the host operating system? (Search for `__linux__` or `linux` in `tcc.h` or `libtcc.c`.)

Write down: your answer for each geography. Some answers will be concrete; some will require a sentence of reasoning.

Problem 5 — Modification: leave a mark [hands-on, 20 min]

This is a small exercise, not a large one.

Open `practice/tcc/tcc.c`. Add a comment — just a comment, not code — somewhere in the file that includes your name, the date, and one sentence about what you learned from reading this code.

The comment should be honest. If you're confused, say so. If something surprised you, say that. If you found something beautiful, say that too.

Rebuild to make sure you didn't accidentally break the syntax.

Write down: the comment, verbatim.

Problem 6 — Zuse and implementation [research, 20 min]

Konrad Zuse designed Plankalkül between 1942 and 1945. It was a genuine high-level language — with loops, conditionals, arrays, and subroutines — but it was never implemented during his lifetime.

Find out: when was Plankalkül first implemented? By whom? What did the implementation look like?

Write down:

- The year and the implementers
 - The gap between design and implementation (in years)
 - What does this case tell you about the relationship between designing a programming language and actually building a compiler for it?
-

Problem 7 — Choose your character [extended research, 45 min]

Choose one person from the phylogeny document. Find one primary source:

- A paper they wrote
- An oral history interview (Computer History Museum, Lex Fridman, YouTube)
- A talk they gave
- A contemporary account by someone who worked with them

Write down:

Who you chose and why

The source: what it is, where you found it, how long it is

The most surprising thing you learned — something not in the phylogeny document

How it changes or deepens the document's account of that person

Minimum 300 words.

Problem 8 — The full circle [synthesis, 20 min]

The phylogeny document ends with this sentence:

"You are at the end of that line. You are not behind. You are not catching up. You are arriving."

You have now read two compilers, traced one function through code generation, modified a living compiler, and followed the history back to 1936. Write a short response — one to three paragraphs — to that sentence.

What does it mean to be at the end of that line? What is different about the way you read code now compared to before this unit?

The problem sets are complete. The reading exercises are a different kind of practice — shorter, more focused, designed to build the specific skill of navigating unfamiliar code. Open `re-01-first-walk.pdf` to begin.